“Compiler Design

Unit 5

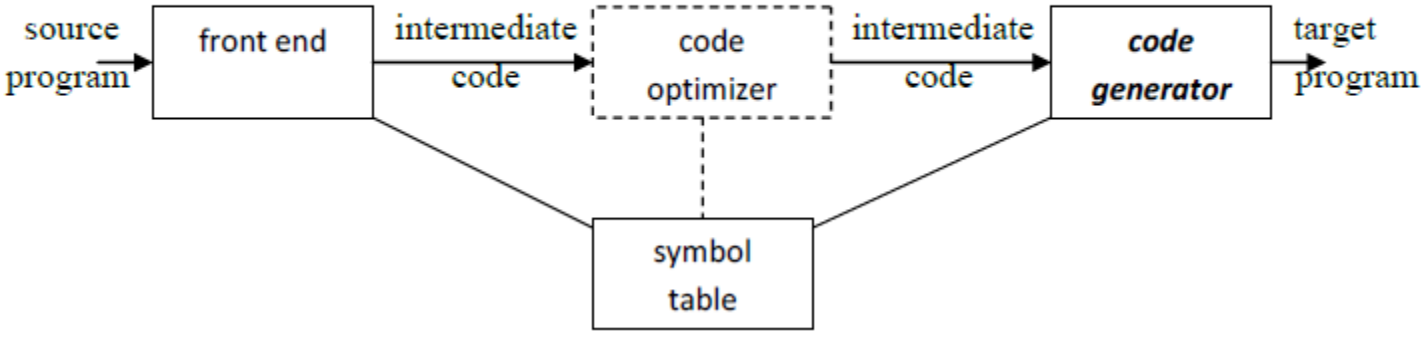
Code Generation

Available on notes.arpanadhikari7.com.np

Introduction

- The final phase in compiler model is the code generator.
- It takes as input an intermediate representation of the source program and produces as output an equivalent target program.
- The code generation techniques presented below can be used whether or not an optimizing phase occurs before code generation.

Position of code generator



Available on notes.arpanadhikari7.com.np

Issues In The Design Of A Code Generator

- The following issues arise during the code generation phase :

1. Input to code generator
2. Target program
3. Memory management
4. Instruction selection
5. Register allocation
6. Evaluation order

Available on notes.arpanadhikari7.com.np

Input to code generator

- The input to the code generation consists of the intermediate representation of the source program produced by front end , together with information in the symbol table to determine run-time addresses of the data objects denoted by the names in the intermediate representation.

- Intermediate representation can be :

a. Linear representation such as postfix notation

b. Three address representation such as quadruples

c. Virtual machine representation such as stack machine code

d. Graphical representations such as syntax trees and dags.

- Prior to code generation, the front end must be scanned, parsed and translated into intermediate representation along with necessary type checking.

- Therefore, input to code generation is assumed to be error-free.

Available on notes.arpanadhikari7.com.np

Target program

- The target program is the final output of the code generator, which can be in the form of absolute machine language, relocatable machine language, or assembly language. Each type of output has its own set of challenges:
- **Absolute Machine Language** is easy to execute but lacks flexibility because it is bound to specific memory locations.
- **Relocatable Machine Language** allows parts of the program to be moved around in memory, making it suitable for linking multiple modules, but it requires a linking loader and has some overhead.
- **Assembly Language** is symbolic and needs an additional step (an assembler) to convert it into machine code, but it makes the code generation process easier.
- Choosing the appropriate form for the target program depends on factors such as the program's needs, execution environment, and whether the program will be linked with other modules.

Available on notes.arpanadhikari7.com.np

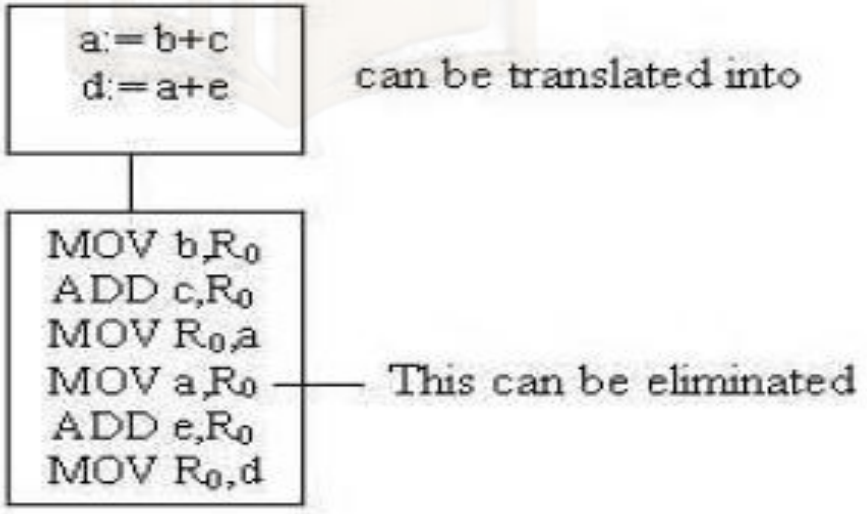
Memory management:

- Names in the source program are mapped to addresses of data objects in run-time memory by the front end and code generator.
- It makes use of symbol table, that is, a name in a three-address statement refers to a symbol table entry for the name.
- Labels in three-address statements have to be converted to addresses of instructions.
- For example, j : goto i generates jump instruction as follows :
- if i<j, a backward jump instruction with target address equal to location of code for quadruple i is generated.
- If i>j, the jump is forward. We must store on a list for quadruple i the location of the first machine instruction generated for quadruple j.
- When i is processed, the machine locations for all instructions that forward jumps to i are filled.

Available on notes.arpanadhikari7.com.np

Instruction selection

- The instructions of target machine should be complete and uniform.
- Instruction speeds and machine idioms are important factors when efficiency of target program is considered.
- The quality of the generated code is determined by its speed and size.
- The former statement can be translated into the latter statement as shown below:



Available on notes.arpanadhikari7.com.np

Register allocation

- Instructions involving register operands are shorter and faster than those involving operands in memory.
- The use of registers is subdivided into two sub problems :
- **Register allocation** – the set of variables that will reside in registers at a point in the program is selected.
- **Register assignment** – the specific register that a variable will reside in is picked.
- Finding an optimal assignment of registers to variables is difficult
- Certain machine requires even-odd register pairs for some operands and results.
- For example , consider the division instruction of the form : D x, y
- where, x – dividend occupies an even register in even/odd register pair whose even register is x
- y – is the divisor
- After division even register holds the remainder odd register holds the quotient

Available on notes.arpanadhikari7.com.np

Evaluation order

- The order in which the computations are performed can affect the efficiency of the target code.
- Some computation orders require fewer registers to hold intermediate results than others.

Available on notes.arpanadhikari7.com.np

Basic Block and Flow Graph

- A basic block is a sequence of consecutive statements in which flow of control enters at the beginning and leaves at the end without any halt or possibility of branching except at the end.

Key Properties of a Basic Block

- **Single Entry Point** – Execution starts only at the first instruction.
- **Single Exit Point** – Control leaves only after the last instruction.
- **No Internal Branches** – No jumps or branches occur within the block.
- **No Branch Targets** – No jumps from outside enter in the middle of the block.

Available on notes.arpanadhikari7.com.np

Basic Block Construction

Algorithm: Partition into basic blocks

Input: A sequence of three-address statements

Output: A list of basic blocks with each three-address statement in exactly one block

Method:

1. We first determine the set of *leaders*, the first statements of basic blocks. The rules we use are of the following:
    a. The first statement is a leader.
    b. Any statement that is the target of a conditional or unconditional goto is a leader.
    c. Any statement that immediately follows a goto or conditional goto statement is a leader.
2. For each leader, its basic block consists of the leader and all statements up to but not including the next leader or the end of the program.

Available on notes.arpanadhikari7.com.np

Consider the following **three-address code** snippet:

1. x = 10
2. y = 20
3. if (x < y) goto 6
4. z = x + y
5. goto 7
6. z = x - y
7. print z

Available on notes.arpanadhikari7.com.np

- **Leaders**: Instructions 1, 3, 4, 6, 7 (since 3 is a branch, 4 follows a branch, 6 is a target, and 7 follows a branch).

Basic Blocks:

- **Block 1**: 1, 2, 3
- **Block 2**: 4, 5
- **Block 3**: 6
- **Block 4**: 7

Available on notes.arpanadhikari7.com.np

```
 1)    i = 1
 2)    j = 1
 3)    t1 = 10 * i
 4)    t2 = t1 + j
 5)    t3 = 8 * t2
 6)    t4 = t3 - 88
 7)    a[t4] = 0.0
 8)    j = j + 1
 9)    if j <= 10 goto (3)
10)    i = i + 1
11)    if i <= 10 goto (2)
12)    i = 1
13)    t5 = i - 1
14)    t6 = 88 * t5
15)    a[t6] = 1.0
16)    i = i + 1
17)    if i <= 10 goto (13)
```

Available on notes.arpanadhikari7.com.np

Find leaders and blocks

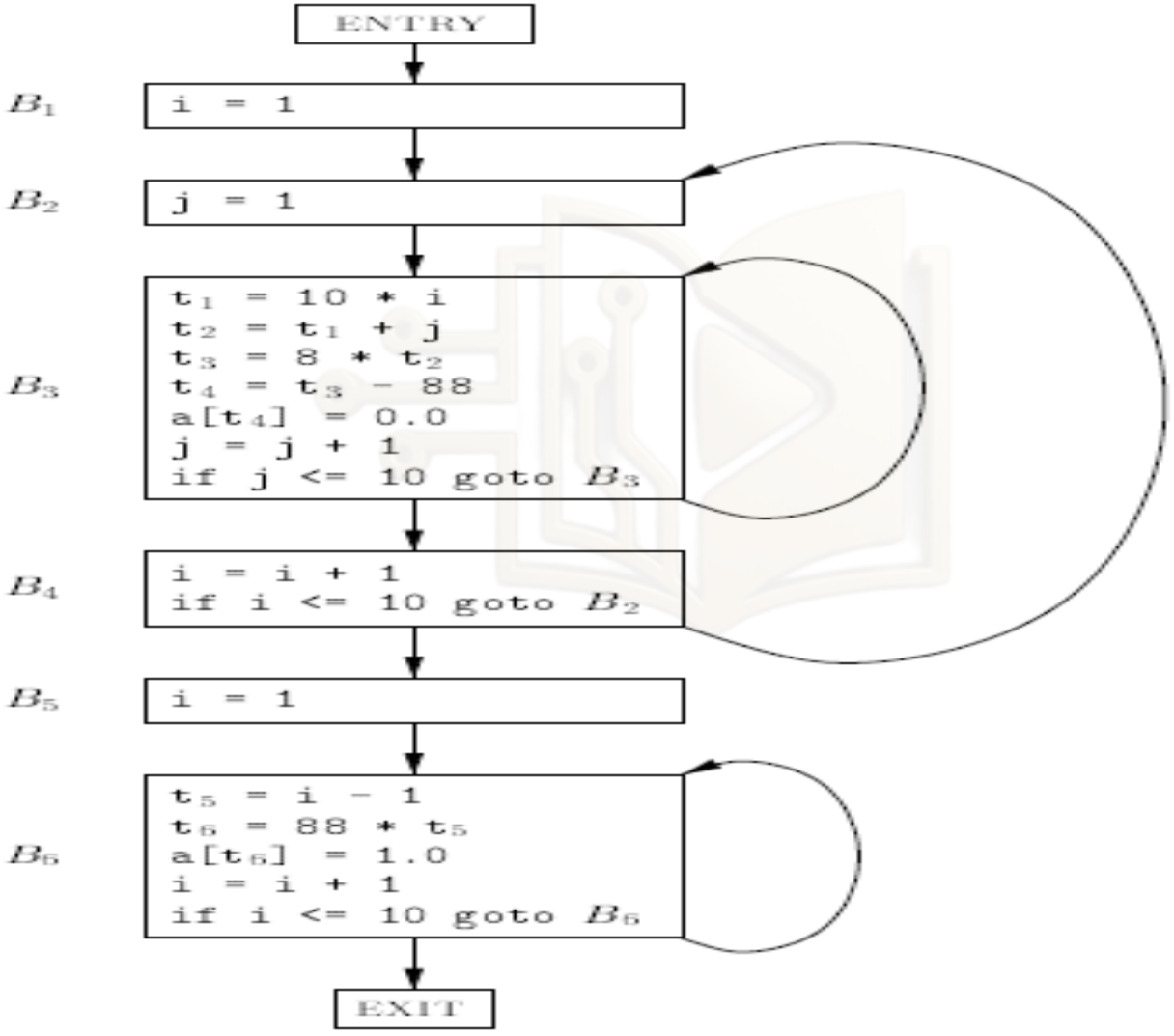
Available on notes.arpanadhikari7.com.np

- A **flow graph** (or **control flow graph, CFG**) is a graphical representation of a program's control structure, where nodes represent **basic blocks** and edges represent **control flow** between them. It is widely used in **compiler optimizations**, **program analysis**, and **testing**.

Structure of a Flow Graph

- **Nodes** → Basic Blocks (sequences of instructions with no branches except at the end).
- **Edges** → Possible execution paths (branches, jumps, fall-through).
- **Start Node** → Entry block (first instruction).
- **End Node** → Exit block (last instruction, may have multiple exits if there are return statements).

Available on notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

Optimization of Basic Block

- Optimizing basic blocks is a crucial step in compiler design, where transformations are applied to a sequence of instructions within a single basic block to improve performance without changing the program's semantics.

- These transformations are useful for improving the quality of code that will be ultimately generated from basic block.

- **There are two types of basic block optimizations:**

1. Structure preserving transformations
2. Algebraic transformations

Available on notes.arpanadhikari7.com.np

Structure-Preserving Transformations

- The Structure-Preserving Transformations in basic Block Optimization include

- **Elimination of common sub-expression**
- **Elimination of Dead Code**
- **Renaming the temporary variables**
- **Interchanging two adjacent independent statements**

Available on notes.arpanadhikari7.com.np

Common Sub expression Elimination (CSE)

- Avoid redundant computations by reusing previously computed results.
- If an expression is computed more than once with the same operands, store the result and reuse it.

Before optimization

```
a = b + c;
d = b + c;  // Redundant computation
```

After optimization

```
temp = b + c;
a = temp;
d = temp;  // Reuses computed value
```

-

Available on notes.arpanadhikari7.com.np

Elimination of Dead Code

- Dead code is defined as that part of the code that never executes during the program execution. So, for optimization, such code or dead code is eliminated.

- Eliminating the dead code increases the speed of the program as the compiler does not have to translate the dead code.

Before Optimization:

- x = y + z; // 'x' is never used later
- a = b + c;

After Optimization:

- a = b + c; // Removed dead assignment 'x = y + z'

Available on notes.arpanadhikari7.com.np

Renaming of Temporary Variables:

- Statements containing instances of a temporary variable can be changed to instances of a new temporary variable without changing the basic block value.

- A statement t:=b+c where t is a temporary name can be changed to u:=b+c where u is another temporary name, and change all uses of t to u.

- In this a basic block is transformed to its equivalent block called normal-form block.

Available on notes.arpanadhikari7.com.np

Interchange of two independent adj acent statements:

- Suppose a block has the following two adjacent statements:

  t1 : = b + c

  t2 : = x + y

- We can interchange the two statements without affecting the value of the block if and only if neither x nor y is t1 and neither b nor c is t2.

Available on notes.arpanadhikari7.com.np

Algebraic Transformation:

- Countless algebraic transformations can be used to change the set of expressions computed by a basic block into an algebraically equivalent set.
- Algebraic transformations can be used to change the set of expressions computed by a basic block into an algebraically equivalent set.

Examples:

- i) x : = x + 0 or x : = x * 1 can be eliminated from a basic block without changing the set of
- expressions it computes.
- ii) The exponential statement x : = y * * 2 can be replaced by x : = y * y.

Available on notes.arpanadhikari7.com.np

Simple Code Generator and peephole optimization

- A code generator generates target code for a sequence of three- address statements and effectively uses registers to store operands of the statements.

- For example: consider the three-address statement **a := b+c**
- It can have the following sequence of codes:
- ADD Rj, Ri Cost = 1 // if Ri contains b and Rj contains c (or)
- ADD c, Ri Cost = 2 // if c is in a memory location (or)
- MOV c, Rj Cost = 3 // move c from memory to Rj and add
- ADD Rj, Ri

Available on notes.arpanadhikari7.com.np

Register and Address Descriptors:

- A register descriptor is used to keep track of what is currently in each registers. The register descriptors show that initially all the registers are empty.

- An address descriptor stores the location where the current value of the name can be found at run time.

Available on notes.arpanadhikari7.com.np

A code-generation algorithm:

- The algorithm takes as input a sequence of three-address statements constituting a basic block. For each three-address statement of the form x : = y op z**, perform the following actions:**

- 1. Invoke a function *getreg to determine the location L where the result of the computation y op z should be stored.*

- 2. Consult the address descriptor for y to determine y′, the current location of y. Prefer the register for y if the value of y is currently both in memory and a register. If the value of y is not already in L, generate the instruction **MOV y′, L to place a copy of y in L.**

Available on notes.arpanadhikari7.com.np

- 3. Generate the instruction OP z′ , L where z′ is a current location of z. Prefer a register to a memory location if z is in both. Update the address descriptor of x to indicate that x is in location L.

- 4. If the current values of y or z have no next uses, are not live on exit from the block, and are in registers, alter the register descriptor to indicate that, after execution of x : = y op z , those registers will no longer contain y or z.

Available on notes.arpanadhikari7.com.np

Example : Generating Code for Assignment Statements:

- The assignment d : = (a-b) + (a-c) + (a-c) might be translated into the following three- address code sequence:

- t : = a – b
- u : = a – c
- v : = t + u
- d : = v + u
- with d live at the end.
- Code sequence for the example is:

Available on notes.arpanadhikari7.com.np

Statements	Code Generated	Register descriptor	Address descriptor
		Register empty	
t : = a - b	MOV a, R_0 SUB b, R0	R_0 contains t	t in R_0
u : = a - c	MOV a , R1 SUB c , R1	R_0 contains t R1 contains u	t in R_0 u in R1
v : = t + u	ADD R_1, R_0	R_0 contains v R_1 contains u	u in R_1 v in R_0
d : = v + u	ADD R_1, R_0 MOV R_0, d	R_0 contains d	d in R_0 d in R_0 and memory

Available on notes.arpanadhikari7.com.np

Generating Code for Indexed Assignments

The table shows the code sequences generated for the indexed assignment statements
a : = b [i] and **a [i] : = b**

Statements	Code Generated	Cost
a : = b[i]	MOV b(R_i), R	2
a[i] : = b	MOV b, a(R_i)	3

Generating Code for Pointer Assignments

The table shows the code sequences generated for the pointer assignments
a : = *p and ***p : = a**

Statements	Code Generated	Cost
a : = *p	MOV *R_p, a	2
*p : = a	MOV a, *R_p	2

Available on notes.arpanadhikari7.com.np

Generating Code for Conditional Statements

Statement	Code
if x < y goto z	CMP x, y CJ< z /* jump to z if condition code is negative */
x : = y +z if x < 0 goto z	MOV y, R_0 ADD z, R_0 MOV R_0,x CJ< z

Available on notes.arpanadhikari7.com.np

Peephole Optimization

- **Peephole Optimization** is a type of code optimization technique used by compilers to improve the performance of generated machine code by examining a small set of instructions (a "peephole") and replacing them with more efficient alternatives.

- A simple but effective technique for improving the target code is peephole optimization, a method for trying to improving the performance of the target program by examining a short sequence of target instructions (called the peephole) and replacing these instructions by a shorter or faster sequence, whenever possible.

Available on notes.arpanadhikari7.com.np

- We shall give the following examples of program transformations that are characteristic of peephole optimizations:
- Redundant-instructions elimination
- Flow-of-control optimizations
- Algebraic simplifications
- Use of machine idioms
- Unreachable Code

Available on notes.arpanadhikari7.com.np

Redundant load and store instructions elimination

- In this optimization, the redundant operations are removed.
- For example, loading and storing values on registers can be optimized.
- If we see the instruction sequence

  LD R0, a

  ST a, R0
- in a target program, we can delete the store instruction because whenever it is executed, the first instruction will ensure that the value of a has already been loaded into register R0.
- If the store instruction had a label, we could not be sure that the first instruction is always executed before the second, so we could not remove the store instruction.
- This means the two instructions have to be in the same basic block for this transformation to be safe.

Available on notes.arpanadhikari7.com.np

Unreachable Code:

- Another opportunity for peephole optimizations is the removal of unreachable instructions.
- An unlabeled instruction immediately following an unconditional jump may be removed.

Before optimization:

```
JMP L1
   MOV R1, R2   ; Unreachable (dead code)
L1:
   ADD R3, R4
```

After peephole optimization:

```
JMP L1
L1:
   ADD R3, R4
```

Available on notes.arpanadhikari7.com.np

- This operation can be repeated to eliminate a sequence of instructions.
- For example, for debugging purposes, a large program may have within it certain segments that are executed only if a variable debug is 1.
- The source code might look like:

Available on notes.arpanadhikari7.com.np

if debug == 1 goto L1

goto L2

L1: print debugging information

L2:

One obvious peephole optimization is to eliminate jumps over jumps.

Thus, no matter what the value of debug, the code sequence above can be replaced by

if debug != 1 goto L2

print debugging information

L2:

Available on notes.arpanadhikari7.com.np

Flow-of-Control Optimizations

- Simple intermediate code-generation algorithms frequently produce jumps to jumps, jumps to conditional jumps, or conditional jumps to jumps.

- These unnecessary jumps can be eliminated in either the intermediate code or the target code by the peephole optimizations.

- We can replace the sequence

Available on notes.arpanadhikari7.com.np

goto L1

L1: goto L2

…..

by the sequence

goto L2

…..

L1: goto L2

If there are now no jumps to L1, then it may be possible to eliminate the statement L1: goto L2 provided it is preceded by an unconditional jump.

Similarly, the sequence

if a < b goto L1

_ _ _ L1: goto L2

can be replaced by the sequence

if a < b goto L2

…..

L1: goto L2

Available on notes.arpanadhikari7.com.np

Algebraic Simplification:

- There is no end to the amount of algebraic simplification that can be attempted through peephole optimization.

- Only a few algebraic identities occur frequently enough that it is worth considering implementing them .For example, statements such as

  x := x+0 Or

  x := x * 1

- Are often produced by straightforward intermediate code-generation algorithms, and they can be eliminated easily through peephole optimization.

Available on notes.arpanadhikari7.com.np

Reduction in Strength:

- Reduction in strength replaces expensive operations by equivalent cheaper ones on the target machine.
- Certain machine instructions are considerably cheaper than others and can often be used as special cases of more expensive operators.
- For example, x^2 is invariably cheaper to implement as x*x than as a call to an exponentiation routine.
- Fixed-point multiplication or division by a power of two is cheaper to implement as a shift.
- Floating-point division by a constant can be implemented as multiplication by a constant, which may be cheaper.
- X2 →X*X

Available on notes.arpanadhikari7.com.np

Use of Machine Idioms:

- The target machine may have hardware instructions to implement certain specific operations efficiently.
- For example, some machines have auto-increment and auto-decrement addressing modes.
- These add or subtract one from an operand before or after using its value.
- The use of these modes greatly improves the quality of code when pushing or popping a stack, as in parameter passing.
- These modes can also be used in code for statements like

  i =i+1.

  i:=i+1→i++

  i:=i-1→i- -

Available on notes.arpanadhikari7.com.np

Register Allocation and Assignment

- Instructions involving only register operands are faster than those involving memory operands.
- On modern machines, processor speeds are often an order
- of magnitude or more faster than memory speeds.
- Therefore, efficient utilization of registers is vitally important in generating good code.
- Various strategies for deciding at each point in a program what values should reside in registers (register allocation) and in which register each value should reside (register assignment).
- One approach to register allocation and assignment is to assign specific values in the target program to certain registers.

Available on notes.arpanadhikari7.com.np

- For example, we could decide to assign base addresses to one group of registers, arithmetic computations to another, the top of the stack to a fixed register, and so on.

Local register allocation

- Register allocation is only within a basic block. It follows top-down approach.
- Assign registers to the most heavily used variables
- Traverse the block
- Count uses
- Use count as a priority function
- Assign registers to higher priority variables first

Available on notes.arpanadhikari7.com.np

Global register allocation:

- Local allocation does not take into account that some instructions (e.g. those in loops) execute more frequently.
- It forces us to store/load at basic block endpoints since each block has no knowledge of the context of others.
- To find out the live range(s) of each variable and the area(s) where the variable is used/defined global allocation is needed.
- In compiler design, global register allocation refers to assigning variables to registers across basic blocks and loops, not just within a single block.
- It's a broader, more optimized form of register allocation compared to **local allocation** (which handles one block at a time).
- Cost of spilling will depend on frequencies and locations of uses.

Available on notes.arpanadhikari7.com.np

Register allocation depends on:

- **Size of live range**

The number of instructions between a variable's definition and last use.

Number of uses/definitions

- Total count of how often a variable is used or redefined in the code.

Frequency of execution

How often the code using the variable is executed (e.g., inside a loop vs. outside).

Number of loads/stores needed.

- If a variable isn't kept in a register, it must be loaded before use and stored after modification.

Cost of loads/stores needed

- Memory access cost depends on architecture (e.g., cache misses, instruction latency).

Available on notes.arpanadhikari7.com.np

Example

```
for (int i = 0; i < 1000; i++) {
    x = x + y;
    z = x * 2;
}
```

x:
Used and defined often,
Has a long live range across the loop,
Lives in a high-frequency region (loop),
Loading/storing each iteration is expensive.
⇒ Strong candidate for register allocation.

z:
Defined but not reused ⇒ Short live range.
⇒ Weaker candidate (could be in memory).

Available on notes.arpanadhikari7.com.np

Usage Counts:

- Usage count is a metric used during register allocation to guide which variables (or temporaries) are most deserving of registers.
- It reflects how often a variable is accessed (read or written) within a given region (typically a basic block, loop, or function).

- An approximate formula for the benefit to be realized from allocating a register to x within a loop L is:

$$\sum_{blocks\ B\ in\ L} (use(x,B) + 2 * live(x,B))$$

- where,
- -use(x, B) is the number of times x is used in B prior to any definition of x;
- -live(x,B) is 1 if x is live on exit from B and is assigned a value in B and 0 otherwise.

Available on notes.arpanadhikari7.com.np

- Example: Consider the basic block in the inner loop in Fig where jump and conditional jumps have been omitted. Assume R0, R1 and R2 are allocated to hold values throughout the loop. Variables live on entry into and on exit from each block are shown in the figure.

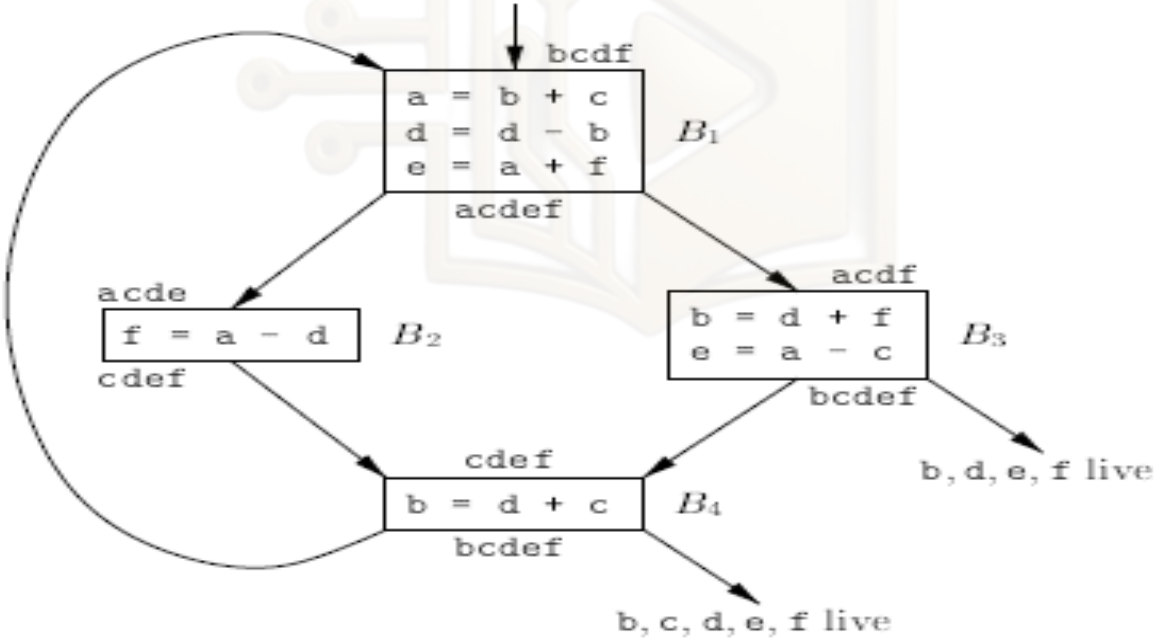


Fig: Flow graph of an inner loop

Available on notes.arpanadhikari7.com.np

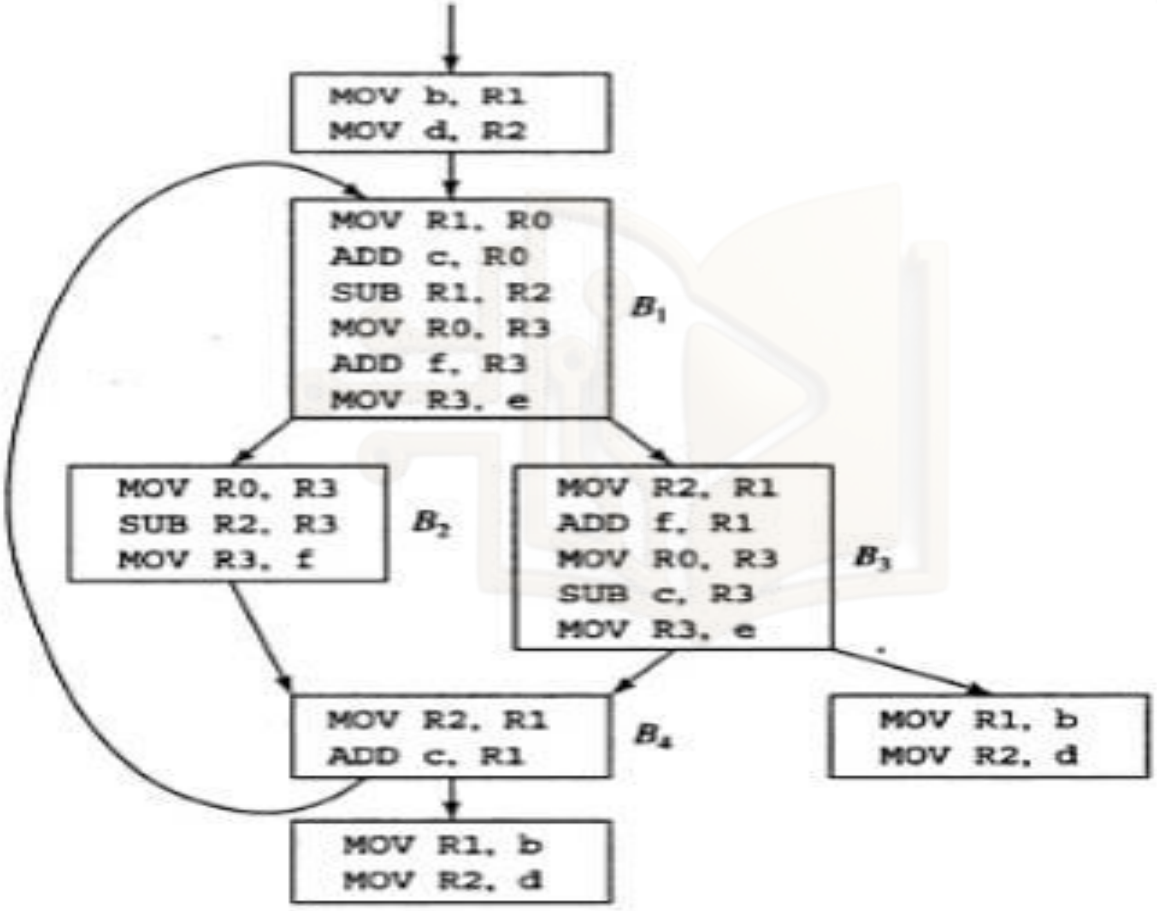


Fig : Code sequence using global register assignment () the assembly code generated from previous figure

Available on notes.arpanadhikari7.com.np

Register Assignment for Outer Loops

- Having assigned registers and generated code for inner loops, we may apply the same idea to progressively larger enclosing loops.
- Inner loops execute more frequently (due to nesting), so variables used in them benefit the most from being held in registers.
- If an outer loop *L1* contains an inner loop L_2, the names allocated registers in L_2 need not be allocated registers in L1 — L2.
- The lifetime of variables in L2 is typically **shorter** than those in L1.
- If we choose to allocate x a register in L2 but not Li, we must load x on entrance to L2 and store x on exit from L2.

Available on notes.arpanadhikari7.com.np

- If variable x is:
- **Assigned a register in L2** but
- **Not in L1** (i.e., it lives in memory in L1), then:
- **Before entering L2**: Load x from memory to its register.
- **After exiting L2**: Store the updated value of x back to memory.

```
for (i = 0; i < N; i++) {      // Outer loop L1
    ... // x might be in memory
    for (j = 0; j < M; j++) {   // Inner loop L2
        ... // x is heavily used, so put x in a register
    }
    ... // back to memory
}
```

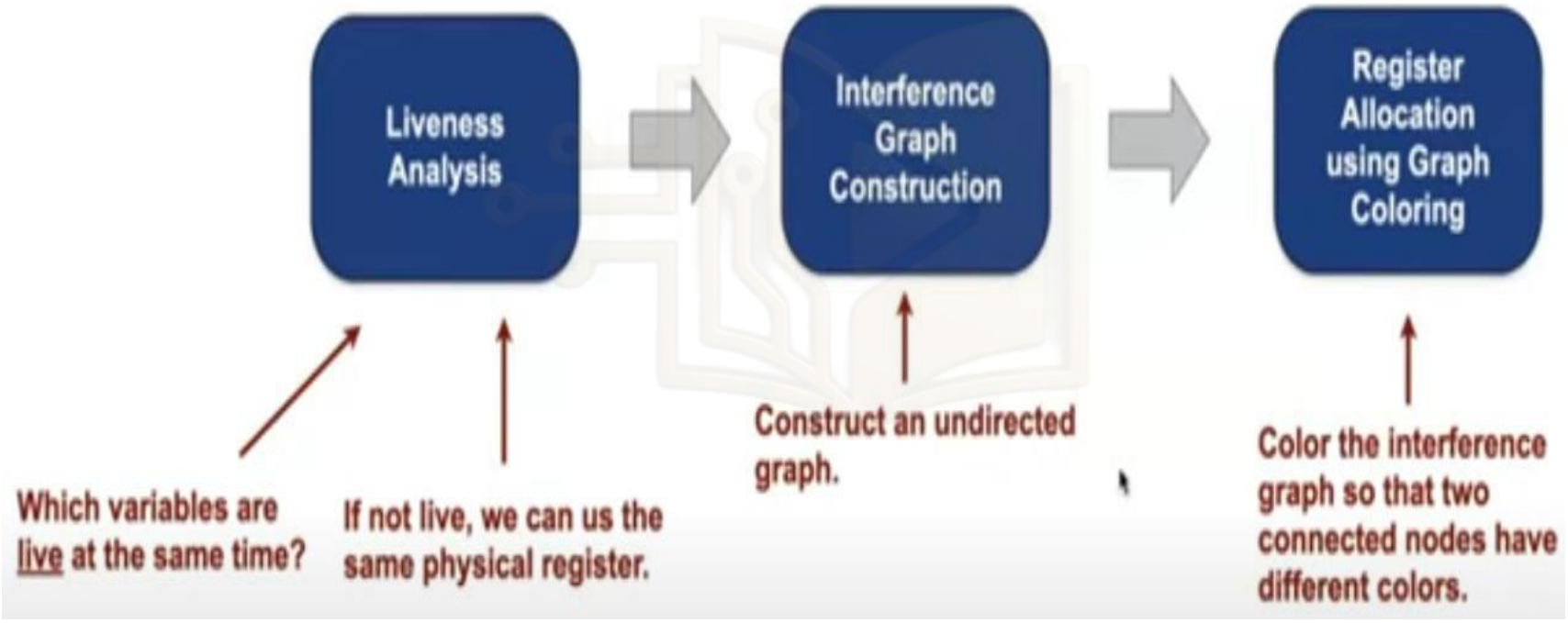
- Compiler **loads x into a register** when entering L2.
- Compiler **stores x back to memory** when exiting L2.

Available on notes.arpanadhikari7.com.np

Register Allocation by Graph Coloring

- When a register is needed for a computation but all available registers are in use, the contents of one of the used registers must be stored *(spilled)* into a memory location in order to free up a register.
- Graph coloring is a simple, systematic technique for allocating registers and managing register spills.
- Register allocation by graph coloring is a widely used global register allocation technique in compilers.
- It models the problem of assigning registers as a **graph coloring problem**, where:
- **Variables** are nodes (vertices),
- **Edges** represent *interference* (i.e., two variables that are live at the same time and **cannot share a register**),
- **Colors** represent **machine registers**.

Available on notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

Basic idea

1. **Identify the Live Range of Each Variable**

- A live range of a variable is the span of code where the variable holds a value that may be used in the future.
- It starts at the definition and ends at the last use.
- Use liveness analysis to determine which variables are live at each program point.

2. **Build the Interference Graph**

- Nodes = Variables
- Edge (A — B) = A and B are live at the same time ⇒ conflict.
- The graph represents interference between variables' live ranges.

3. **Color the Graph with k Colors**

- Try to assign colors (registers) to each node so that:
  - No two adjacent nodes (interfering variables) have the same color.
  - k = number of available physical registers on the target machine.

Available on notes.arpanadhikari7.com.np

- Assume 3 variables: a, b, c, and 2 registers: R1, R2

Liveness info:

- a and b live together
- b and c live together
- a and c do not interfere (no edge)

Interference Graph:

- a --- b --- c

This graph **can be colored with 2 colors**:

- a → R1
- b → R2
- c → R1

Available on notes.arpanadhikari7.com.np

Dynamic Programming Code Generation

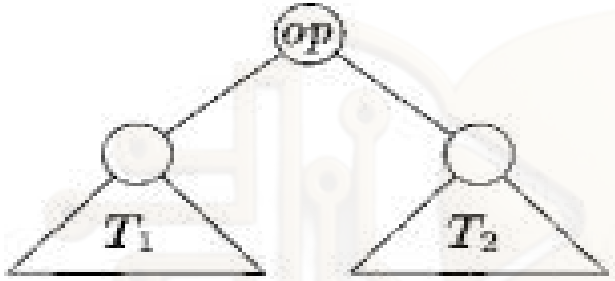
- A dynamic programming algorithm is used to extend the class of machines for which optimal code can be generated from expressions trees in linear time.
- This algorithm works for a broad class of register machines with complex instruction sets.
- Such an algorithm is used to generate code for machines with *r* interchangeable registers *R0, R1, ..., Rr-1* and load, store and operation instructions.
- Throughout this article, we will assume that every instruction has a cost of one unit however a dynamic programming algorithm works even with instructions having their own costs.

Available on notes.arpanadhikari7.com.np

Contiguous evaluation.

- The algorithm divides the problem of generating optimal code for an expression into sub problems of generating optimal code for sub expressions of the given expression.
- For example consider an expression *E* in the form of *E1* + *E2*.
- To obtain an optimal program of *E*, we combine optimal program for E1 and E2 followed by the code to evaluate the + operator.
- Sub problems for generating code for E1 and E2 are also solved in a similar manner.
- The program produced by the dynamic programming algorithm has the following property, it evaluates expression *E* = *E1 op E2* "contiguously".
- Let's look at the syntax tree T for E to have a better idea:

Available on notes.arpanadhikari7.com.np



- *T1* and *T2* are trees for *E1* and *E2* respectively.

- We say that a program *P* evaluates tree *T* *contiguously* if it first, evaluates the sub trees of *T* that need to be computed in memory followed by evaluating the remainder of *T* by using previously computed values from memory.

Available on notes.arpanadhikari7.com.np

- Let's look at an example of a *noncontiguous* evaluation: *P* first evaluates part of *T1* leaving the value in a register, next it evaluates *T2* then it returns to evaluate *T1*.

- For the register machine in this article, we prove that for any given machine-language program *P* that evaluates an expression tree *T*, there is an equivalent program P' such that;

- It's cost is less than that of P.
- It uses fewer registers than P.
- P' evaluates the tree contiguously.

Available on notes.arpanadhikari7.com.np

The algorithm.

- The algorithm has three phases, we assume the machine has three registers;

1. First we compute bottom-up for each node n of the expression tree T an array C of costs whereby its ith component $C[i]$ is the optimal cost of computing the subtree S rooted at n into a register, assuming i registers are available for the computation for $1 \leq i \leq r$.

2. Traverse T using the cost vectors to determine subtrees of T that must be computed into memory.

3. Traverse each tree using the cost vectors and associate instructions to generate the target code. The code for the subtrees that are computed into memory locations is generated first.

Available on notes.arpanadhikari7.com.np